

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación Gráfica, Visión Computacional y Multimedia				
TÍTULO DE LA PRÁCTICA:	<i>Transformaciones Geometricas</i>				
NÚMERO DE PRÁCTICA:	02	AÑO LECTIVO:	2026	NRO. SEMESTRE:	2026A
FECHA DE PRESENTACIÓN	03/05/2026	HORA DE PRESENTACION	23:59		
INTEGRANTE (s): Chahua Charca, Luis Ayben				NOTA:	
DOCENTE(s): <i>Rene Alonso Nieto Valencia</i>					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS 1. Controlador del jugador Para la entrada de datos, se construye el playerController que permite que el jugador se mueva con WASD ademas, de saltar con espacio, tomar un objeto con E e interactuar con una máquina con Q 1.1. Movimiento básico <pre> CSharp void Move() { rb.linearVelocity = new Vector2(moveInput * moveSpeed, rb.linearVelocity.y); } void Jump() { rb.linearVelocity = new Vector2(rb.linearVelocity.x, jumpForce); } </pre> 1.2 Trygrabb y Release Object: Estos 2 metodos son utilizados por la tecla "E" para tomar uno de los 3 objetos que se pueden tomar en el nivel. 

CSharp

```
void TryGrab()
{
    Collider2D hit = Physics2D.OverlapCircle(transform.position, grabRange,
grabbableLayer);

    if (hit == null) return; // evita el error

    grabbedObject = hit.gameObject;
    grabbedRb = grabbedObject.GetComponent<Rigidbody2D>();

    if (grabbedRb == null) return; // seguridad extra

    MachineSlot[] slots = Object.FindObjectsByType<MachineSlot>();

    foreach (var s in slots)
    {
        if (s.storedObject == grabbedObject.transform)
        {
            s.storedObject = null;
            break;
        }
    }

    // Desactivar física
    grabbedRb.bodyType = RigidbodyType2D.Kinematic;
    grabbedRb.constraints = RigidbodyConstraints2D.None; // 🔥 clave
    grabbedRb.freezeRotation = true; // solo mientras lo cargas

    // IGNORAR COLISIÓN
    Collider2D playerCol = GetComponent<Collider2D>();
    Collider2D objectCol = grabbedObject.GetComponent<Collider2D>();

    if (playerCol != null && objectCol != null)
    {
        Physics2D.IgnoreCollision(playerCol, objectCol, true);
    }
}

void ReleaseObject()
{
}
```

```
if (grabbedObject == null) return;

MachineSlot slot = FindClosestSlot();

if (slot != null && slot.storedObject == null && slot.playerInRange)
{
    grabbedObject.transform.SetPositionAndRotation(
        slot.transform.position,
        Quaternion.identity
    );

    grabbedRb.bodyType = RigidbodyType2D.Kinematic;
    grabbedRb.constraints = RigidbodyConstraints2D.FreezeAll;

    Collider2D playerCol = GetComponent<Collider2D>();
    Collider2D objectCol = grabbedObject.GetComponent<Collider2D>();

    if (playerCol != null && objectCol != null)
    {
        Physics2D.IgnoreCollision(playerCol, objectCol, false);
    }

    slot.storedObject = grabbedObject.transform;

    grabbedObject = null;
    grabbedRb = null;

    return;
}

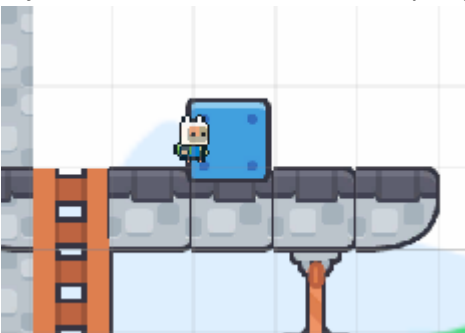
// normal
Collider2D playerCol2 = GetComponent<Collider2D>();
Collider2D objectCol2 = grabbedObject.GetComponent<Collider2D>();

if (playerCol2 != null && objectCol2 != null)
{
    Physics2D.IgnoreCollision(playerCol2, objectCol2, false);
}

grabbedRb.bodyType = RigidbodyType2D.Dynamic;
grabbedRb.freezeRotation = false;
```

```
grabbedObject = null;
grabbedRb = null;
}
```

Establece objetos nulos que actuarán como detectores, para ver si un elemento está en el rango para poder ser tomado, ese elemento calcula la distancia y si resulta true, y presionan la tecla, el jugador recogerá el objeto, es decir, desactiva su física y se pega a él y lo seguirá



1.3 TryUseMachine: llama a la función para activar una palanca para que haga una de las 2 cosas: Escalar y Rotar las figuras

CSharp

```
void TryUseMachine()
{
    Collider2D hit = Physics2D.OverlapCircle(transform.position,
interactRange, machineLayer);

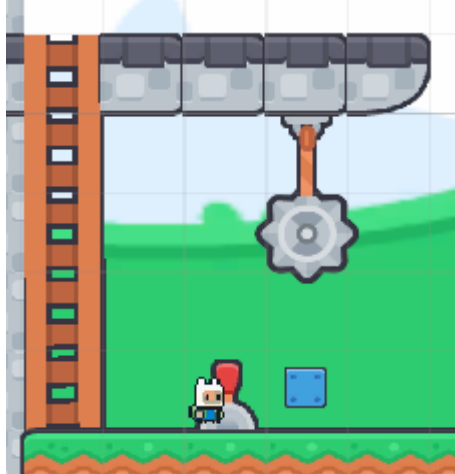
    if (hit != null)
    {
        IMachine machine = hit.GetComponentInParent<IMachine>();

        if (machine != null)
        {
            machine.Activate();
        }
    }
}
```

2. Máquinas de rotación y escalado

dentro del juego existen 2 “Máquinas” que al interactuar con las palancas, estos transformaran el objeto en uno con diferente rotación o escalado

2.1 Escalator Machine



CSharp

```
public void Activate()
{
    if (isBusy) return;
    Debug.Log("Objeto en slot: " + slot.storedObject);
    if (slot.storedObject != null)
    {
        Transform obj = slot.storedObject;

        obj.rotation = Quaternion.Euler(0, 0, obj.eulerAngles.z + 90f);

        StartCoroutine(LeverAnimation());
    }
}

IEnumerator LeverAnimation()
{
    isBusy = true;

    sr.sprite = leverDown;

    yield return new WaitForSeconds(1f);
}
```

```
sr.sprite = leverUp;

isBusy = false;
}
```

Ambos trabajan en base a un slot, donde la maquina entra y que se controla con este script

CSharp

```
public Transform storedObject;
public bool playerInRange = false;

void OnTriggerEnter2D(Collider2D other)
{
    if (other.CompareTag("Player"))
    {
        playerInRange = true;
    }
}

void OnTriggerExit2D(Collider2D other)
{
    if (other.CompareTag("Player"))
    {
        playerInRange = false;
    }
}

void OnTriggerStay2D(Collider2D other)
{
    if (storedObject == null) return;

    if (other.transform != storedObject) return;

    Rigidbody2D rb = storedObject.GetComponent<Rigidbody2D>();
    if (rb == null) return;

    // solo cuando ya casi está quieto
    if (rb.linearVelocity.magnitude > 0.1f) return;

    // centrar suavemente (snap final)
    storedObject.position = Vector3.Lerp(
```

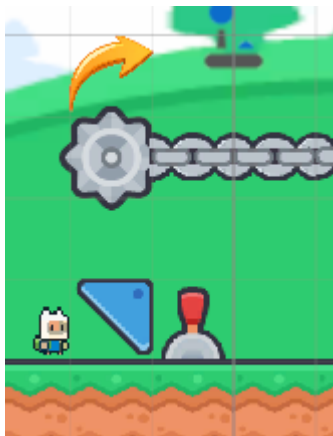
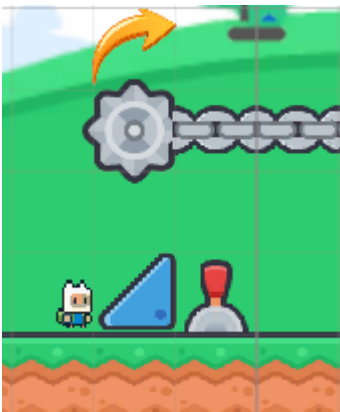
```

        storedObject.position,
        transform.position,
        0.2f
    );

    // opcional: si ya está casi perfecto → fijarlo totalmente
    if (Vector2.Distance(storedObject.position, transform.position) < 0.01f)
    {
        storedObject.position = transform.position;
    }

```

2.2 Rotate Machine: Este es igual al otro, ya que depende de un slot, y gira el objeto puesto en ese slot.



CSharp

```

public void Activate()
{
    if (isBusy) return;
    Debug.Log("Objeto en slot: " + slot.storedObject);
    if (slot.storedObject != null)
    {
        Transform obj = slot.storedObject;

        obj.rotation = Quaternion.Euler(0, 0, obj.eulerAngles.z + 90f);

        StartCoroutine(LeverAnimation());
    }
}

IEnumerator LeverAnimation()
{
    isBusy = true;

```

```

sr.sprite = leverDown;

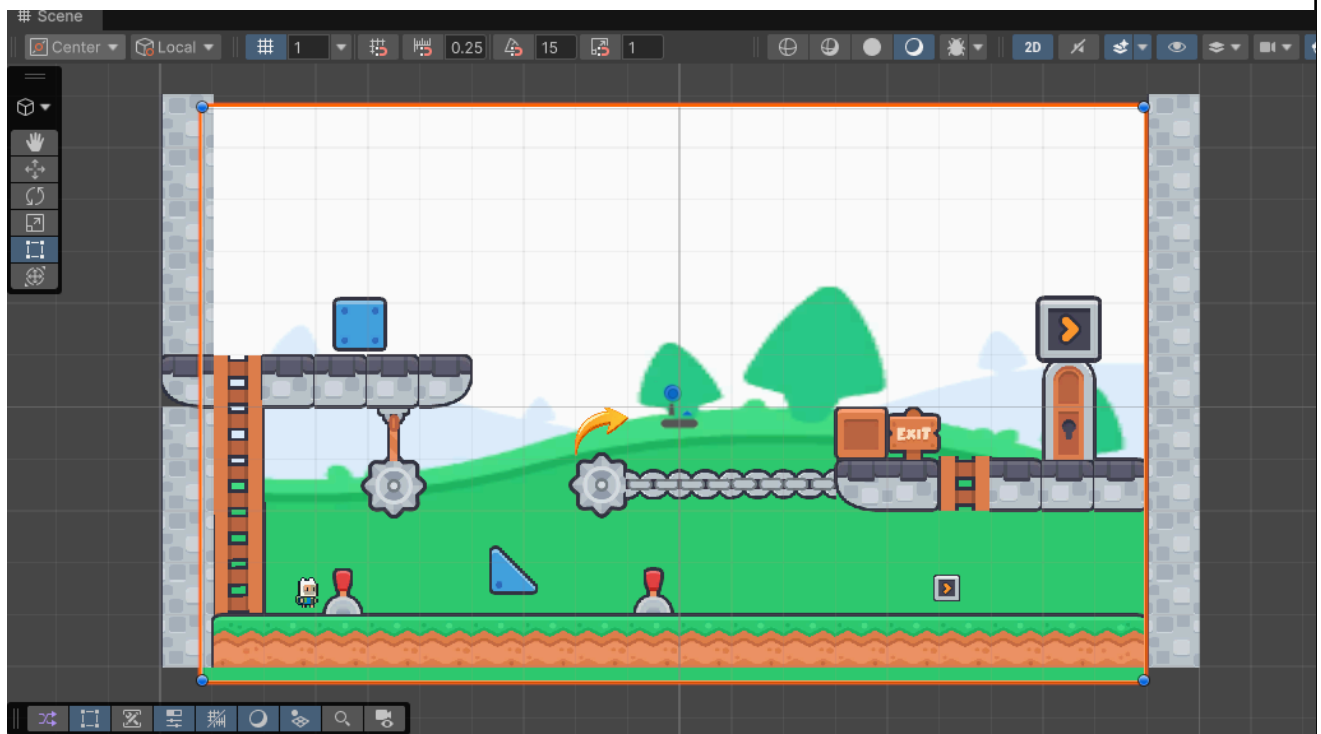
yield return new WaitForSeconds(1f);

sr.sprite = leverUp;

isBusy = false;
}

```

Resultado Final



II. SOLUCIÓN DEL CUESTIONARIO

III. CONCLUSIONES



UNIVERSIDAD NACIONAL DE SAN AGUSTIN
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA



Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación

Aprobación: 2022/03/01

Código: GUIA-PRLE-001

Página: 9

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA